BANKING FRAUD DETECTION USING PYTHON & MACHINE LEARNING

Prepared by: Kiran Kansla

Tools & Technologies

- Python
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- Jupyter Notebook

Project Type: Machine Learning • Banking Analytics • Fraud Detection

TABLE OF CONTENTS:

Project Overview

Background

Digital banking has significantly increased the number of online financial transactions. While digital services provide convenience, they have also created opportunities for increasingly sophisticated fraudulent activities. Financial institutions require intelligent fraud detection systems capable of identifying suspicious transactions in real time while minimizing disruption to legitimate customers.

This project develops a machine learning–based fraud detection system using a synthetic banking transaction dataset that simulates realistic customer behavior and fraud scenarios. The objective is to identify fraudulent transactions with high accuracy while providing actionable insights into the factors contributing to fraud.

Project Objectives

1. Analyze transaction behavior using exploratory data analysis.
2. Identify key factors associated with fraudulent transactions.
3. Build and compare multiple machine learning classification models.
4. Evaluate model performance using standard classification metrics.
5. Provide business recommendations for improving fraud prevention.

Dataset Description

The dataset contains 10,000 banking transactions with 18 features representing transaction details, customer behavior, risk indicators, and fraud labels.

Target Variable

fraud_flag

0 = Legitimate Transaction

1 = Fraudulent Transaction

Key Features

Feature	Description
transaction_amount	Monetary value of the transaction
login_attempts	Number of login attempts before transaction
device_risk_score	Risk score assigned to the customer's device
transfer_frequency	Number of transfers performed
anomaly_score	AI-generated anomaly probability
account_age_days	Age of customer account
transaction_time_hour	Hour of transaction
avg_monthly_balance	Customer's average monthly balance
daily_transaction_count	Daily transaction frequency
geo_distance_km	Geographic distance from normal location
session_duration_minutes	Duration of banking session
transaction_velocity_score	Speed of transaction activity
payment_channel	Mobile App, Web Banking, ATM, POS
authentication_type	OTP, Two-Factor Authentication, Password, Biometric
suspicious_ip_flag	Suspicious IP indicator
international_transaction_flag	Domestic or International
fraud_flag	Fraud Label

Data Preprocessing

The dataset was examined to ensure quality before model development.

The preprocessing workflow included:

1. Loading the dataset into Pandas.
2. Inspecting column names and data types.
3. Reviewing summary statistics.
4. Checking for missing values.
5. Checking duplicate records.
6. Encoding categorical variables.
7. Splitting features and target variables.
8. Creating training and testing datasets using an 80:20 ratio.

No major data quality issues were observed, allowing model development without extensive cleaning..

Exploratory Data Analysis

Several visualizations were created to understand fraud patterns and customer behavior.

Major analyses included:

1. Fraud distribution
2. Transaction amount distribution
3. Authentication type analysis
4. Payment channel analysis
5. Correlation heatmap
6. Anomaly score distribution
7. Boxplots
8. Feature relationships

Key Findings

1. Fraud represented 12.5% of all transactions.
2. Legitimate transactions accounted for 87.5%.
3. Mobile App transactions contributed the highest transaction volume.
4. OTP authentication was the most frequently used authentication method.
5. Fraud and legitimate transactions exhibited similar transaction amount distributions, indicating that transaction amount alone is not a reliable fraud indicator.
6. The anomaly score demonstrated the strongest relationship with fraudulent activity.

Feature Engineering

To improve predictive performance, feature engineering techniques were applied.

The following steps were performed:

1. Label encoding for categorical variables.

2. Feature selection.
3. Train-test split.
4. Preparation of numerical features for machine learning algorithms.

No synthetic features were required because the dataset already included engineered behavioral indicators such as:

1. Device Risk Score
2. Transaction Velocity Score
3. AI Anomaly Score
4. Geographic Distance

These variables significantly enhanced model performance.

Model Development

Three supervised machine learning algorithms were implemented.

Logistic Regression

A baseline classification model used to establish benchmark performance.
Advantages

- Fast
- Interpretable
- Suitable for binary classification

Decision Tree

A rule-based classifier capable of capturing nonlinear relationships.
Advantages

- Easy to interpret
- Handles nonlinear patterns
- Minimal preprocessing

Random Forest

An ensemble learning algorithm combining multiple decision trees.
Advantages

- Higher accuracy
- Reduced overfitting
- Better generalization
- Robust feature importance estimation

Random Forest was selected as the final model due to superior overall performance.

Model Evaluation

The models were evaluated using five classification metrics.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	95.00%	84.00%	77.60%	80.80%
Decision Tree	94.45%	78.14%	77.20%	77.67%
Random Forest	**95.25%**	**82.70%**	**78.40%**	**80.49%**

Confusion Matrix Analysis

The Logistic Regression confusion matrix produced:

True Negatives: 1713

True Positives: 194

False Positives: 37

False Negatives: 56

Interpretation

1. The model correctly classified the majority of legitimate transactions.
2. Only 37 legitimate transactions were incorrectly flagged as fraud.
3. Fifty-six fraudulent transactions remained undetected.
4. Overall classification performance demonstrates strong predictive capability suitable for banking fraud detection.

ROC Curve Analysis

The ROC Curve evaluates classification performance across different decision thresholds.

Results

Logistic Regression ROC-AUC : **0.979**

Random Forest ROC-AUC: **0.975**

Interpretation

Both models achieved excellent discrimination between fraudulent and legitimate transactions. An AUC value close to 1.0 indicates outstanding predictive performance.

Feature Importance Analysis

Random Forest identified the most influential predictors.

Top Features

1. Anomaly Score
2. Geographic Distance
3. Average Monthly Balance
4. Account Age
5. Transaction Amount
6. Device Risk Score
7. Transaction Velocity Score

Among all variables, Anomaly Score contributed the highest importance (approximately 0.62), making it the strongest predictor of fraudulent activity.

Model Comparison

Random Forest demonstrated the highest overall predictive performance.

Reasons for selecting Random Forest:

1. Highest Accuracy
2. Highest Precision
3. Strong Recall
4. Excellent ROC-AUC
5. Better generalization
6. Robust feature importance interpretation

Although Logistic Regression produced a slightly higher ROC-AUC score, Random Forest achieved the best overall balance across all evaluation metrics and was therefore selected as the final production model.

Technical Conclusion

This project successfully developed a machine learning solution capable of detecting fraudulent banking transactions with high predictive accuracy. Through exploratory data analysis, feature engineering, and model comparison, Random Forest emerged as the most effective classification algorithm.

The results demonstrate that behavioral indicators such as anomaly score, geographic distance, and device risk score provide substantially greater predictive power than transaction amount alone. The developed workflow can serve as a foundation for real-time fraud detection systems in digital banking environments.

Future Improvements

Future enhancements could further improve model performance and real-world applicability.

Recommended improvements include:

1. Hyperparameter optimization using GridSearchCV or RandomizedSearchCV.
2. Cross-validation to improve model robustness.
3. Addressing class imbalance using SMOTE or class weighting.
4. Testing advanced algorithms such as XGBoost, LightGBM, and CatBoost.
5. Developing real-time fraud scoring pipelines.
6. Integrating streaming transaction data for continuous monitoring.
7. Deploying the model through REST APIs for production use.
8. Building an interactive monitoring dashboard using Power BI or Streamlit.